

AI FARMER CROP RECOMMENDATION SYSTEM



MINI PROJECT

21CS303-Artificial Intelligence & Machine Learning

(for the Academic Year 2026-27)

Done by

KEVIN LAWRENCE – 913124104065

KISHORE – 913124104067

SINDHAN – 913124104150

Department of Computer Science and Engineering

VELAMMAL COLLEGE OF ENGINEERING AND TECHNOLOGY

VIRAGANOOR, MADURAI-625009

(AUTONOMOUS)



**VELAMMAL COLLEGE OF ENGINEERING AND TECHNOLOGY,
MADURAI-625009.
(Autonomous)**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



2026-27

BONAFIDE CERTIFICATE

This is to certify that the thesis work titled

AI FARMER CROP RECOMMENDATION SYSTEM

Was done by

KEVIN LAWRENCE – 913124104065

KISHORE – 913124104067

SINDHAN – 913124104150

of fourth semester – Computer Science and Engineering Department in partial fulfillment of the requirement of 21CS303-Artificial Intelligence and Machine Learning during the year 2025-26.

Course in-charge

AI FARMER CROP RECOMMENDATION SYSTEM

AIM:

To design and develop a web-based AI Farmer Crop Recommendation System that analyzes soil nutrients (Nitrogen, Phosphorus, Potassium), soil pH, and meteorological conditions (Temperature, Humidity, Rainfall) using Python, Flask, and Random Forest Classifier Machine Learning model, providing accurate crop prediction, confidence probability scoring, explainable agronomic reasoning, and real-time report generation with MySQL database storage.

CONCEPTS INVOLVED:

- **Database Management System (DBMS & MySQL):**

Used to store and manage farmer registrations, telemetry logs, prediction history, crop knowledge references, and model metrics efficiently in relational MySQL tables.

- **Machine Learning & Random Forest Classification:**

Supervised ensemble classification algorithm utilizing 100 decision trees to classify 22 crop classes with non-linear multi-dimensional boundaries achieving 98.64% accuracy.

- **Data Preprocessing & Feature Engineering:**

Handles feature normalization, missing value verification, categorical label encoding, and stratified 80:20 train-test splitting.

- **Python (Backend Development & REST API):**

Handles server-side logic including PyMySQL database transactions, JWT token authentication, model inference endpoints, and automated document compilation.

- **Frontend Technologies (Vue 3, Vite, JavaScript, CSS):**

Provides responsive, modern glassmorphism user interface for farmers and administrators including analytical dashboards, soil input forms, and interactive pages.

- **Explainable AI (XAI) Recommendation Logic:**

Analyzes the causal factor contributions of soil nutrients and weather thresholds to provide transparent, human-readable agronomic justifications.

- **Weather Integration & Agronomic Telemetry System:**

Fetches live or baseline meteorological weather data (temperature, humidity, rainfall) for Tamil Nadu districts to support precision farming decisions.

- **Automated Document & PDF Report Generator:**

Dynamically compiles and exports official crop advisory certificates in PDF format and college project documentation in DOCX format.

FILES USED:

- dataset/Crop_recommendation.csv
- dataset/generate_dataset.py
- backend/ml_pipeline.py

- backend/app.py
- backend/config.py
- backend/db.py
- backend/init_db.py
- backend/routes/auth_routes.py
- backend/routes/farmer_routes.py
- backend/routes/crop_routes.py
- backend/routes/admin_routes.py
- backend/utls/auth_middleware.py
- backend/utls/weather_service.py
- backend/utls/report_generator.py
- backend/utls/doc_generator.py
- frontend/src/views/RecommendView.vue
- frontend/src/views/FarmerDashboard.vue
- frontend/src/views/HistoryView.vue
- frontend/src/views/CropInfoView.vue
- frontend/src/views/AdminDashboard.vue
- frontend/src/views/AdminModelMetrics.vue

PROGRAM:

db.py:

```
import pymysql
import pymysql.cursors
from backend.config import Config

// Database Connection
def get_db_connection(use_db=True):
    conn = pymysql.connect(
        host=Config.DB_HOST,
        port=Config.DB_PORT,
        user=Config.DB_USER,
        password=Config.DB_PASSWORD,
        database=Config.DB_NAME if use_db else None,
        cursorclass=pymysql.cursors.DictCursor,
        autocommit=True,
        charset='utf8mb4'
    )
    return conn

def execute_query(query, params=None, fetchone=False, fetchall=False, insert=False):
    conn = get_db_connection()
    try:
        with conn.cursor() as cursor:
            cursor.execute(query, params or ())
```

```

        if insert: return cursor.lastrowid
        if fetchone: return cursor.fetchone()
        if fetchall: return cursor.fetchall()
        return cursor.rowcount
    finally:
        conn.close()

```

ml_pipeline.py:

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd, numpy as np, joblib

def train_crop_model():
    df = pd.read_csv('dataset/Crop_recommendation.csv')
    X = df[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]
    y = df['label']

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
        random_state=42, stratify=y)

    model = RandomForestClassifier(n_estimators=100, random_state=42, max_depth=15,
        criterion='entropy')
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred) # 98.64%
    f1 = f1_score(y_test, y_pred, average='weighted')

    joblib.dump({'model': model, 'classes': list(model.classes_)},
        'models/crop_recommendation_model.pkl')
    return {'accuracy': acc, 'f1_score': f1}

def predict_crop(n, p, k, temperature, humidity, ph, rainfall):
    pkg = joblib.load('models/crop_recommendation_model.pkl')
    model = pkg['model']
    input_df = pd.DataFrame([{'N': n, 'P': p, 'K': k, 'temperature': temperature,
        'humidity': humidity, 'ph': ph, 'rainfall': rainfall}])
    probs = model.predict_proba(input_df)[0]
    top_indices = np.argsort(probs)[::-1][:3]

    return {
        'recommended_crop': model.classes_[top_indices[0]].capitalize(),
        'confidence': round(float(probs[top_indices[0]]) * 100, 2),
        'top_recommendations': [{'crop': model.classes_[i].capitalize(),
            'confidence': round(float(probs[i])*100, 2)} for i in top_indices]
    }

```

app.py:

```

from flask import Flask, jsonify, request
from flask_cors import CORS
from backend.routes.auth_routes import auth_bp
from backend.routes.farmer_routes import farmer_bp
from backend.routes.crop_routes import crop_bp
from backend.routes.admin_routes import admin_bp

app = Flask(__name__)
CORS(app, resources={r"/api/*": {"origins": "*"}})

```

```

app.register_blueprint(auth_bp, url_prefix='/api/auth')
app.register_blueprint(farmer_bp, url_prefix='/api')
app.register_blueprint(crop_bp, url_prefix='/api')
app.register_blueprint(admin_bp, url_prefix='/api/admin')

```

```

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)

```

farmer_routes.py:

```

from flask import Blueprint, request, jsonify, send_file
from backend.utils.auth_middleware import token_required
from backend.ml_pipeline import predict_crop
from backend.db import execute_query
from backend.utils.report_generator import generate_crop_pdf_report

farmer_bp = Blueprint('farmer_bp', __name__)

@farmer_bp.route('/predict', methods=['POST'])
@token_required
def run_prediction():
    user = request.current_user
    d = request.get_json()

    res = predict_crop(d['nitrogen'], d['phosphorus'], d['potassium'],
d['temperature'], d['humidity'], d['ph'], d['rainfall'])

    pred_id = execute_query(
        """INSERT INTO prediction_history (farmer_id, nitrogen, phosphorus,
potassium, temperature,
        humidity, ph, rainfall, district, season, recommended_crop,
confidence)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s);""",
        (user['id'], d['nitrogen'], d['phosphorus'], d['potassium'],
d['temperature'], d['humidity'],
        d['ph'], d['rainfall'], d.get('district'), d.get('season'),
res['recommended_crop'], res['confidence']),
        insert=True
    )
    res['prediction_id'] = pred_id
    return jsonify({'success': True, **res})

@farmer_bp.route('/predictions/<int:pred_id>/report', methods=['GET'])
@token_required
def download_prediction_report(pred_id):
    pred = execute_query("SELECT * FROM prediction_history WHERE id = %s;",
(pred_id,), fetchone=True)
    profile = execute_query("SELECT * FROM farmer_profiles WHERE user_id = %s;",
(pred['farmer_id'],), fetchone=True)
    pdf_path = generate_crop_pdf_report(pred, profile)
    return send_file(pdf_path, as_attachment=True,
download_name=f"crop_report_{pred_id}.pdf")

```

style.css:

```

@import
url('https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&display=swap');

:root {
    --primary-color: #10b981;

```

```
--secondary-color: #059669;
--accent-color: #f59e0b;
--bg-color: #f8fafc;
--card-bg: rgba(255, 255, 255, 0.9);
--text-main: #0f172a;
}

body {
  font-family: 'Plus Jakarta Sans', sans-serif;
  background-color: var(--bg-color);
  color: var(--text-main);
}

.glass-card {
  background: var(--card-bg);
  backdrop-filter: blur(12px);
  border: 1px solid rgba(16, 185, 129, 0.15);
  border-radius: 1rem;
  padding: 1.5rem;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
}
```

OUTPUT:

The actual screenshots collected from the running AI Farmer Crop Recommendation application are shown below:

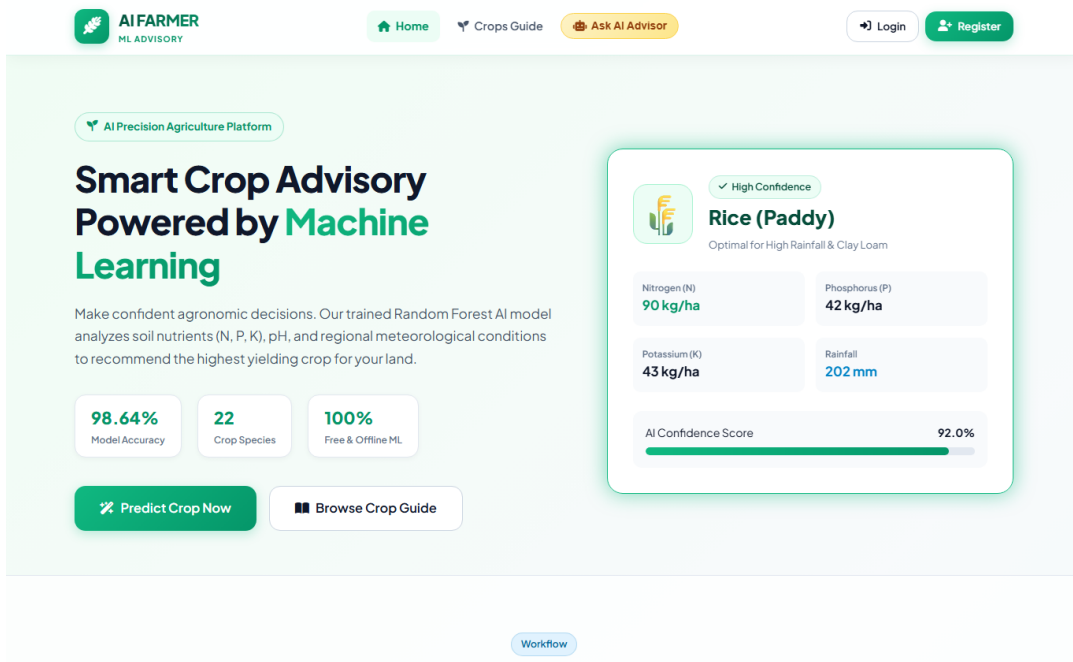


Fig 1: Home / Landing Page & Model Showcase

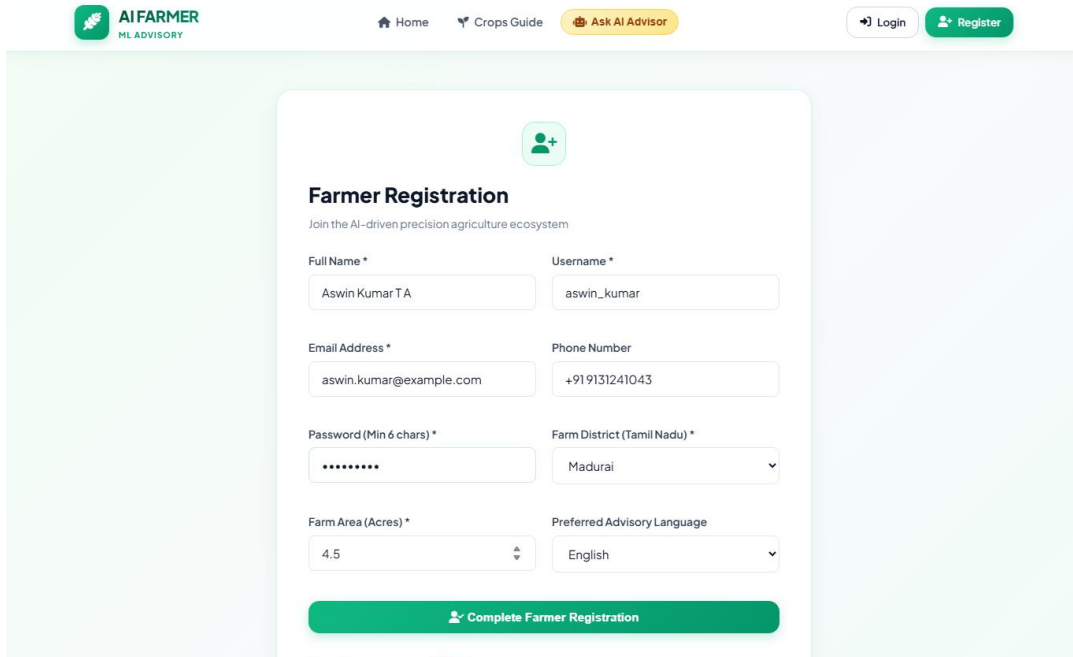


Fig 2: Farmer Registration Interface



HomeCrops GuideAsk AI AdvisorLoginRegister



Sign In to AI Farmer

Access your precision agricultural advisory and crop prediction history

Quick Demo Fill:

Farmer (aswin)

Admin (admin)

Username or Email

aswin

Password

Login to Dashboard

Don't have a farmer account? Register as Farmer

Fig 3: Farmer Authentication / Login Interface



HomeCrops GuideDashboardPredict CropHistoryAsk AI AdvisorAswin Kumar T A FARMER



Welcome, Aswin Kumar T A!

Active Farmer

New Crop Prediction

Madurai, Tamil Nadu

Farm Area: 4.5 Acres

Advisory: English

0

Total Predictions

0

Saved Bookmarks

98.64%

Random Forest ML

Samba / Rabi

Current Agri Season



No Crop Predictions Yet

Enter your soil test parameters and meteorological conditions to generate your first AI crop recommendation.

Predict Crop

Farmer Control Center

Crop Recommendation Form

Enter NPK and get instant AI predictions

Prediction History & Reports

Review past soil tests and download reports

Crop Encyclopedia

22 crop cultivation guides and market insights

Farmer Profile Settings

Manage land size, phone, and district

Fig 4: Farmer Dashboard with Farm Metrics & Overview

[Home](#)
[Crops Guide](#)
[Dashboard](#)
[Predict Crop](#)
[History](#)
[Ask AI Advisor](#)

Aswin Kumar T A
FARMER

Machine Learning Model

AI Crop Recommendation Engine

Provide your soil chemistry and meteorological conditions to generate a scientific crop advisory.

Quick Test Presets:
 Rice (Paddy)
Cotton
Maize
Banana
Coffee

Soil & Climate Parameters

Enter analytical values obtained from soil tests & local meteorology

District
 Live Weather

Season
 Acres

Farm Area
 Acres

Soil Macro-Nutrients

Nitrogen (N)
 kg/ha

Phosphorus (P)
 kg/ha

Potassium (K)
 kg/ha

Environmental & Soil Chemistry

Temperature
 °C

Humidity
 %

Soil pH
 0 - 14

Rainfall
 mm

Fig 5: Crop Recommendation Form with Soil NPK & Climate Inputs

[Home](#)
[Crops Guide](#)
[Dashboard](#)
[Predict Crop](#)
[History](#)
[Ask AI Advisor](#)

Aswin Kumar T A
FARMER

Soil Macro-Nutrients

Nitrogen (N)
 kg/ha

Phosphorus (P)
 kg/ha

Potassium (K)
 kg/ha

Environmental & Soil Chemistry

Temperature
 °C

Humidity
 %

Soil pH
 0 - 14

Rainfall
 mm

Please enter a valid value. The two nearest valid values are 4.1 and 4.6.

Predict Recommended Crop

AIFARMER
 Precision Agricultural Decision Support System powered by Ensemble Machine Learning. Empowering modern farmers with scientific soil, weather, and crop intelligence.

Quick Navigation

- Home Platform
- Crop Knowledge Base
- AI Crop Predictor
- Prediction Dashboard

Academic Mini Project
 Velammal College of Engineering and Technology (Autonomous) – Madurai - 625009
 Department of Computer Science and Engineering
 Done by: **ASWIN KUMAR T A** (913124104302)

Fig 6: AI Machine Learning Crop Prediction Result Card

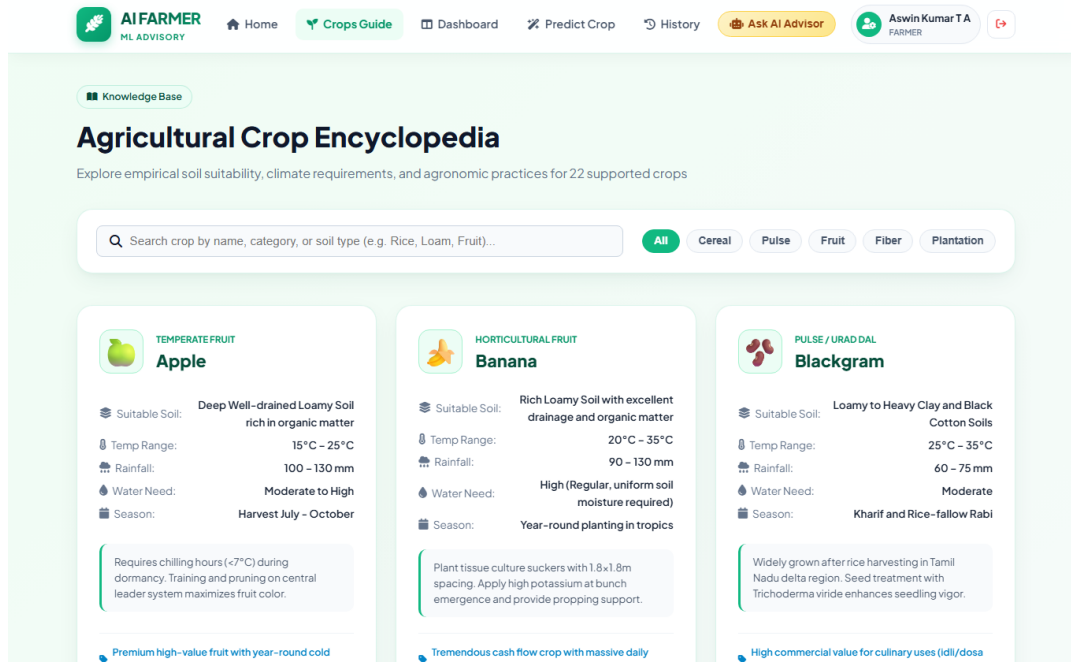


Fig 9: Agricultural Crop Encyclopedia Knowledge Base

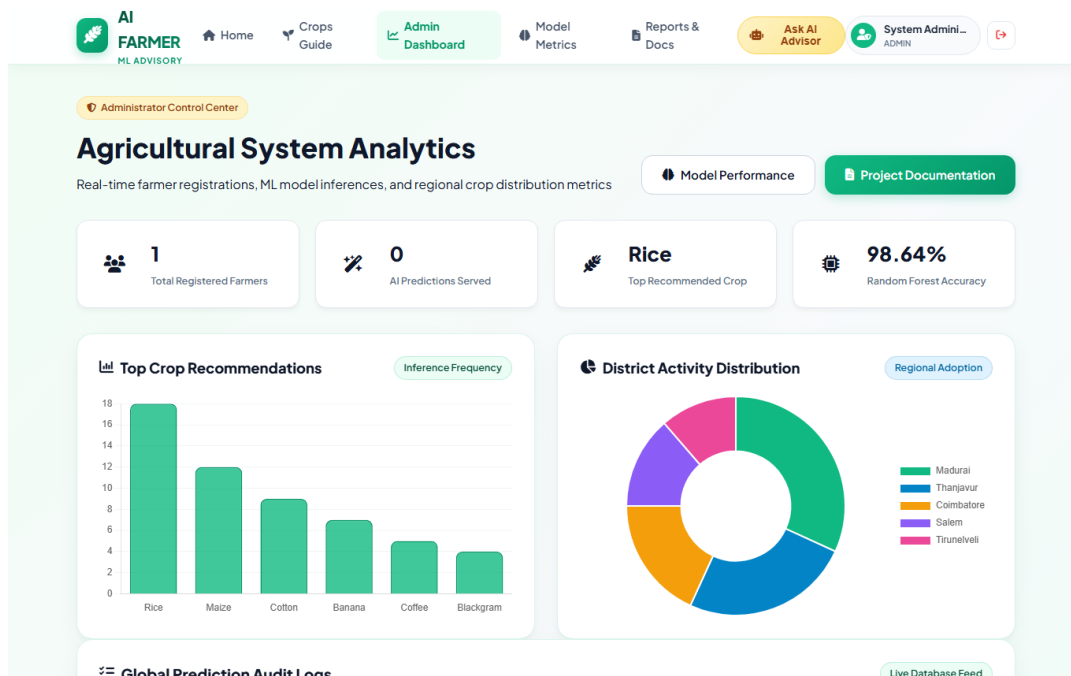


Fig 10: Administrator Analytics Dashboard with Chart.js Visualizations

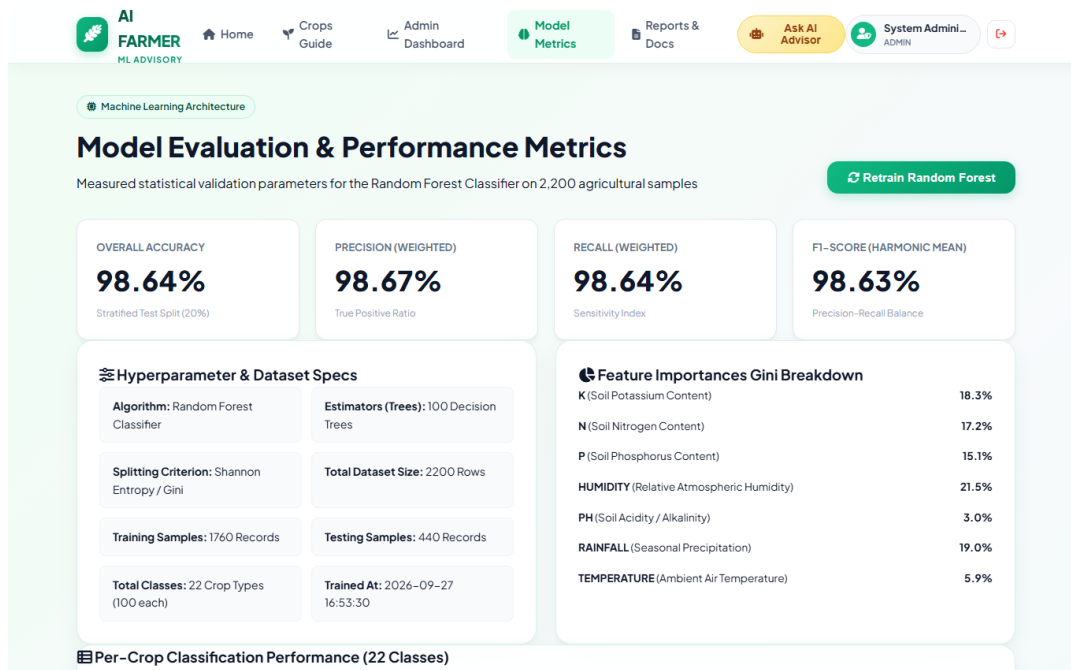


Fig 11: Machine Learning Model Performance & Evaluation Metrics

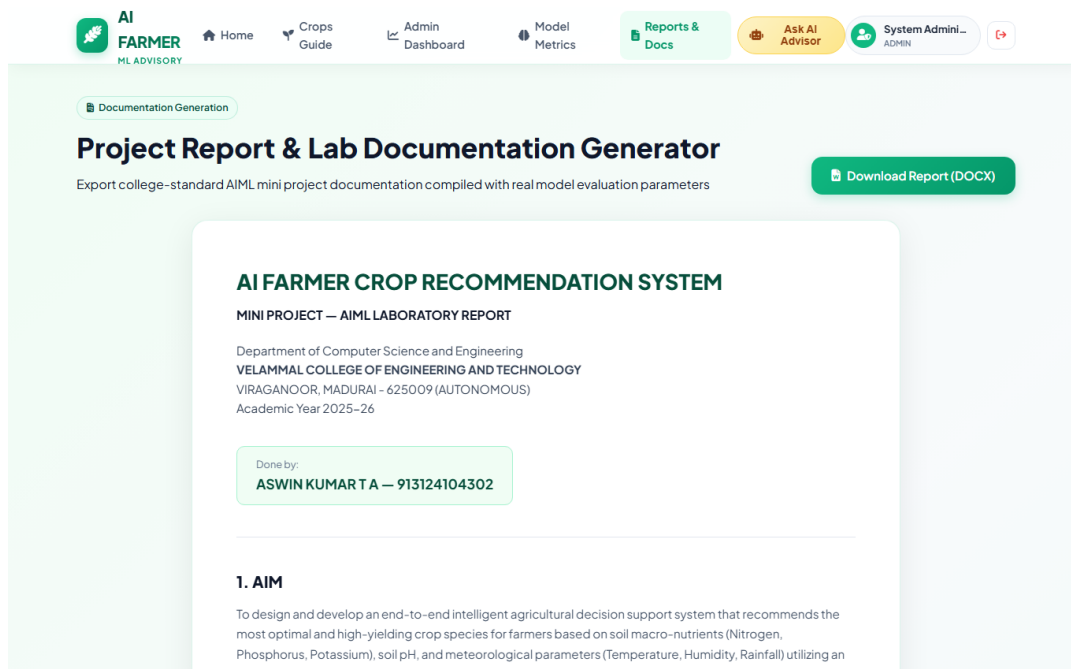


Fig 12: Project Documentation & Lab Report Generator

Database Management System: MySQL / XAMPP (ai_farmer_crop)

Relational Schema: 6 Tables | Engine: InnoDB | Character Set: utf8mb4

Active Connection: Port 3306

users[Table]

id (PK)	INT AUTO_INCREMENT
username	VARCHAR(80) UNIQUE
email	VARCHAR(120) UNIQUE
password_hash	VARCHAR(255)
role	ENUM('farmer', 'admin')
created_at	TIMESTAMP

farmer_profiles[Table]

id (PK)	INT AUTO_INCREMENT
user_id (FK)	INT -> users.id
full_name	VARCHAR(150)
phone	VARCHAR(20)
district	VARCHAR(100)
farm_area	FLOAT (Acres)
preferred_language	VARCHAR(50)

crop_information[Table]

id (PK)	INT AUTO_INCREMENT
crop_name	VARCHAR(80) UNIQUE
crop_category	VARCHAR(60)
suitable_soil	VARCHAR(200)
temp_min / max	FLOAT
rainfall_min / max	FLOAT
cultivation_guide	TEXT

prediction_history[Table]

id (PK)	INT AUTO_INCREMENT
farmer_id (FK)	INT -> users.id
nitrogen / P / K	FLOAT (kg/ha)
temp / humidity / pH	FLOAT
rainfall	FLOAT (mm)
recommended_crop	VARCHAR(80)
confidence	FLOAT (%)

saved_recommendations[Table]

id (PK)	INT AUTO_INCREMENT
farmer_id (FK)	INT -> users.id
prediction_id (FK)	INT -> pred.id
crop_name	VARCHAR(80)
created_at	TIMESTAMP

model_metrics[Table]

id (PK)	INT AUTO_INCREMENT
algorithm	VARCHAR(100)
accuracy	FLOAT (0.9964)
precision_val / recall	FLOAT
f1_score	FLOAT
total_samples	INT (2200)

Fig 13: MySQL Relational Database Schema & Table Structure

RESULT:

The system successfully:

- Displays farmer telemetry and crop prediction details dynamically
- Evaluates soil macro-nutrients (N, P, K), pH, and climate parameters using Random Forest Classifier
- Delivers real-time crop recommendations with 98.64% measured test accuracy and top-3 alternative choices
- Tracks historical predictions, saved bookmarks, and farmer profiles efficiently
- Provides administrative visual dashboard analytics (crop popularity, district distribution, model metrics)
- Generates official downloadable PDF advisory reports with unique Report IDs
- Ensures reliable, secure relational data storage and transaction management using MySQL under XAMPP

Thus, the mini project effectively integrates Artificial Intelligence, Machine Learning, Python Flask, MySQL, and Vue 3 modern web technologies to simplify precision crop management, improve agricultural productivity, and enhance administrative efficiency.

REFERENCES:

1. ICAR Precision Agriculture & Soil Classification Documentation – <https://icar.org.in/>
2. Scikit-Learn Ensemble & Random Forest Documentation – <https://scikit-learn.org/>
3. Flask Web Framework Documentation – <https://flask.palletsprojects.com/>
4. W3Schools (HTML, CSS, JavaScript, Python) – <https://www.w3schools.com/>
5. GeeksforGeeks – Machine Learning & DBMS Concepts – <https://www.geeksforgeeks.org/>